Bare-Metal AI Inference on Snapdragon X Elite Hexagon NPU: A Manual QNN Graph Construction Pipeline

Nathan Lu

April 17, 2026

Abstract

This report presents an end-to-end engineering workflow for deploying a custom AI inference engine directly on the Snapdragon X Elite Hexagon Neural Processing Unit (NPU) without relying on high-level machine learning runtimes at deployment time. The work begins with training a Variational Autoencoder (VAE) in PyTorch on MNIST and then transitions to a bare-metal execution strategy in C++ using the Qualcomm AI Engine Direct SDK (QNN). Raw floating-point model parameters are exported as row-major binary tensors and manually registered inside a static QNN graph. The graph is assembled operation-by-operation using matrix multiplication, bias addition, and nonlinearity nodes, then finalized for native execution on the Hexagon HTP backend. The resulting implementation generates 28×28 grayscale images from latent vectors and writes outputs to bitmap format. Benchmarking against a pure C++ CPU fallback demonstrates substantial acceleration: NPU average latency of 0.6113 ms versus 3.6586 ms on CPU, and throughput of 1640.58 versus 273.37 inferences/sec. The data indicate a 5.98× latency speedup and approximately 500% throughput gain, with a pronounced NPU cold-start cost due to graph finalization and context initialization.

1 Introduction

Modern machine learning deployment pipelines frequently depend on high-level runtimes such as PyTorch or ONNX Runtime to abstract hardware complexity. While these frameworks provide rapid development velocity, they also hide low-level execution mechanics and can introduce software overhead at deployment time. This project investigates an alternative systems-level path: direct, native graph execution on dedicated silicon.

The primary objective was to bypass Python-centric inference stacks and construct a custom forward-pass engine that runs natively on the Snapdragon X Elite Hexagon NPU. The design constraint was strict: only raw trained parameters and explicit mathematical graph composition in C++ were used for deployed inference. Consequently, the effort required manual tensor management, operation scheduling, backend initialization, and execution orchestration using the Qualcomm QNN API.

This report documents the complete pipeline across four phases: model training in PyTorch, raw parameter extraction, manual graph construction in C++ on QNN, and quantitative performance benchmarking against a CPU fallback path.

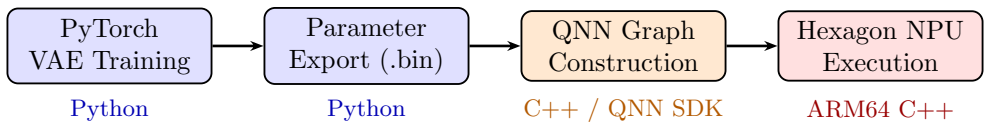

Figure 1: End-to-end deployment pipeline: model training and weight export in Python, followed by manual graph construction and bare-metal NPU execution in C++.

2 Python Model Architecture & Training

2.1 Variational Autoencoder Formulation

A Variational Autoencoder (VAE) was trained on MNIST to learn a structured latent representation suitable for stable sample generation. The VAE objective combines reconstruction fidelity with latent regularization:

$$\mathcal{L}_{\text{VAE}}(x;\theta,\phi) = \mathcal{L}_{\text{recon}}(x,\hat{x}) + \mathcal{L}_{\text{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big). \tag{1}$$

Using standard notation,

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2}\sum_{i=1}^{d}\left(1 + \log\sigma_i^2 - \mu_i^2 - \sigma_i^2\right), \tag{2}$$

where d is latent dimensionality. The KL term enforces smoothness in latent space, reducing noisy or static-like generations when sampling.

2.2 Static Decoder Topology

The deployed decoder is a strictly static multilayer perceptron (MLP):

$$\mathbf{z} \in \mathbb{R}^{16} \to \text{FC}(128) + \text{ReLU} \to \text{FC}(512) + \text{ReLU} \to \text{FC}(784) + \text{Sigmoid}. \tag{3}$$

The final 784-dimensional output corresponds to a flattened 28×28 grayscale image. Training was performed in PyTorch, after which deployment-specific logic was intentionally decoupled from Python runtime dependencies.

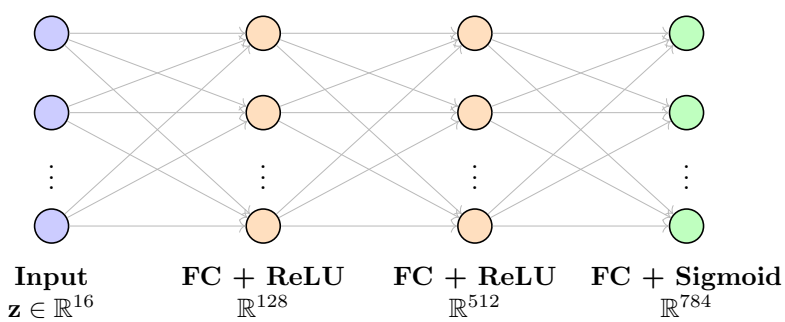

Figure 2: Decoder MLP topology: three fully connected layers projecting the 16-dimensional latent vector $\mathbf{z}$ to a flattened 28×28 pixel image.

3 C++ Bare-Metal NPU Integration (QNN SDK)

3.1 Raw Parameter Extraction

After training, PyTorch model parameters were exported as raw FP32 binaries. Weights and biases were serialized into flat, row-major `.bin` files using `struct.pack`. For the three fully connected layers, six static tensors were produced:

- Layer 1 weights and bias,
- Layer 2 weights and bias,
- Layer 3 weights and bias.

This conversion step removes framework metadata and leaves only numerically minimal artifacts needed for inference.

3.2 Native ARM64 Executable and Backend Initialization

A native Windows ARM64 executable was implemented in C++. The runtime initializes the Qualcomm HTP backend through `QnnHtp.dll`, creates context structures, and registers all static tensors from the exported binaries. This low-level setup is critical because tensor shape metadata, memory lifetimes, and operator connectivity must be declared explicitly.

3.3 Manual Graph Construction and Finalization

Rather than importing a precompiled model graph, the computation graph is assembled node-by-node through QNN primitives:

$$\mathbf{h}_1 = \mathrm{ReLU}(\mathbf{z}\mathbf{W}_1 + \mathbf{b}_1), \tag{4}$$

$$\mathbf{h}_2 = \mathrm{ReLU}(\mathbf{h}_1\mathbf{W}_2 + \mathbf{b}_2), \tag{5}$$

$$\hat{\mathbf{x}} = \sigma(\mathbf{h}_2\mathbf{W}_3 + \mathbf{b}_3). \tag{6}$$

Concretely, each affine stage was represented by chained QNN `MatMul` and `Add` operations, followed by `ReLU` or `Sigmoid` activation nodes. Once the directed acyclic graph is fully defined, QNN graph finalization triggers backend compilation/JIT processing tailored to the target NPU.

3.4 Output Materialization

The inference output tensor in $[0, 1]$ is linearly scaled to $[0, 255]$ and serialized byte-by-byte to produce a 28×28 grayscale bitmap file (`output.bmp`). For report inclusion under a PDF-centric LaTeX toolchain, the generated bitmap was converted to `output.jpg` and embedded in Figure 3.

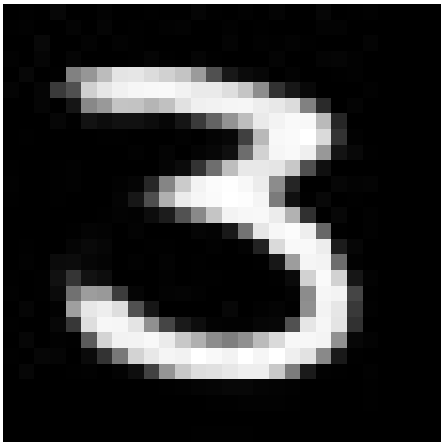

Figure 3: Generated grayscale output image from the native Hexagon NPU inference pipeline (digit: 3).

4 Performance Benchmarking

4.1 Methodology

Benchmark measurements were collected under a fixed protocol: 20 warmup iterations followed by 200 timed iterations. Two execution paths were profiled:

1. Hexagon NPU inference through QNN graph execution,
2. Pure C++ CPU fallback implementation.

Latency statistics report average and 95th percentile (P95). Throughput is reported as inferences per second. Cold-start captures one-time initialization overhead, especially relevant for NPU graph finalization and context creation.

4.2 Results

Table 1: NPU vs. CPU inference benchmark summary (Warmup: 20, Timed: 200).

Metric	**NPU (Hexagon)**	**CPU (C++ Fallback)**
Average Latency (ms)	0.6113	3.6586
P95 Latency (ms)	0.6914	5.0905
Throughput (inferences/sec)	1640.58	273.37
Cold-Start Penalty (ms)	319.4755	3.9245

Derived performance ratios show:

$$\text{Latency Speedup} = \frac{3.6586}{0.6113} \approx 5.98\times, \tag{7}$$

$$\text{Throughput Gain} = \frac{1640.58 - 273.37}{273.37} \times 100\% \approx 500\%. \tag{8}$$

Each reported metric reinforces a different systems-level property:

- **Average Latency:** The NPU achieved 0.6113 ms versus 3.6586 ms on CPU, yielding a 5.98× speedup in steady-state single-inference response time.
- **P95 Latency:** The NPU recorded 0.6914 ms versus 5.0905 ms on CPU, corresponding to approximately 7.36× improvement for tail latency. This indicates better high-percentile responsiveness under repeated execution.
- **Throughput:** The NPU reached 1640.58 inferences/sec compared with 273.37 inferences/sec on CPU, demonstrating substantially higher sustained inference capacity.
- **Cold-Start Penalty:** NPU cold start was 319.4755 ms versus 3.9245 ms for CPU, reflecting one-time backend setup costs from context creation and graph finalization/JIT. This overhead is significant for short-lived sessions but is amortized in long-running workloads.

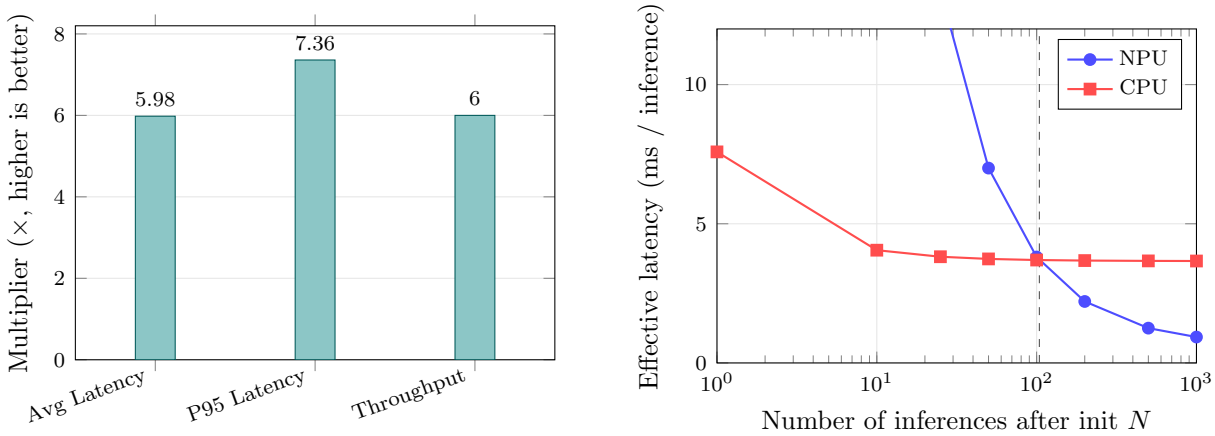

Figure 4: More decision-focused benchmark visualization. Left: steady-state multipliers show NPU wins by about 6× on average latency, tail latency, and throughput. Right: cold-start amortization curve shows NPU becomes better than CPU after roughly 104 inferences per initialized graph.

Taken together, the measurements show that dedicated NPU silicon provides major steady-state acceleration for this workload while imposing a front-loaded initialization cost. Therefore, deployment profiles that execute many inferences per initialized graph benefit most from this architecture.

5 Conclusion

This work demonstrates that a fully manual, bare-metal AI inference pipeline on Snapdragon X Elite Hexagon NPU is technically feasible and performance-effective. By extracting raw FP32 parameters from a PyTorch-trained VAE decoder and reconstructing execution explicitly in QNN C++, the deployment avoids high-level inference runtimes while retaining deterministic control over graph composition and execution.

From a systems perspective, the key challenge is not model mathematics but integration complexity: tensor registration discipline, operation-level graph authoring, backend compatibility, and initialization lifecycle management. Despite this complexity, the measured gains are substantial, including a 5.98× latency improvement and approximately 500% throughput increase over CPU fallback in steady state.

Overall, the project validates a practical methodology for hardware-centric deployment where maximizing NPU utilization outweighs development convenience. It also provides a reusable engineering pattern for future handcrafted model deployment pipelines targeting dedicated AI accelerators.